# Travel Companion

AI Restaurant Finder

---

**Proiect Complet — Toate Variantele de Execuție**

Algoritmi Paraleli și Distribuiți

---

| | |
|---|---|
| **S6 (Baseline)** | Varianta Secvențială |
| **Paralel #1** | Async I/O (`asyncio` + `ThreadPoolExecutor`) |
| **Paralel #2** | Multiprocessing (`multiprocessing.Pool`) |


| | |
|---|---|
| **Student** | Stoian Cristian Georgian |
| **Grupă** | CR3.2A, Calculatoare Română |
| **Data** | Mai 2026 |


---

Python 3.12 | asyncio | multiprocessing | Geoapify | Overpass/OSM | Foursquare

# Cuprins


**1 Cerințe și Tema Proiectului** **3**
1.1 Descrierea Proiectului . . . 3
1.2 Obiectivul Principal . . . 3
1.3 Tehnologii Utilizate . . . 3
1.4 Surse de Date API . . . 3
1.5 Geocodare și Rezolvarea Coordonatelor . . . 4

**2 Arhitectura Sistemului** **4**
2.1 Strategy Pattern . . . 4
2.2 Structura Proiectului . . . 5
2.3 Modelul de Date . . . 5
2.4 Formula de Scoring . . . 6

**3 Specificații Mașină de Test** **6**

**4 Varianta Secvențială (S6 — Baseline)** **6**
4.1 Descrierea Fluxului . . . 6
4.2 Implementare . . . 7
4.3 Rezultate Experimentale — Modul Secvențial . . . 8

**5 Varianta Paralelă #1 — Async I/O** **8**
5.1 Principiu și Justificare . . . 8
5.2 Motivare Tehnică: De ce ThreadPoolExecutor? . . . 9
5.3 Implementare . . . 9
5.4 Proprietăți ale Implementării . . . 10

**6 Varianta Paralelă #2 — Multiprocessing** **10**
6.1 Principiu și Justificare . . . 10
6.2 Constrângeri Tehnice: Pickle . . . 11
6.3 Implementare . . . 11
6.4 Reconstituirea Timing-ului pentru Faza I/O . . . 12

**7 Comparație Arhitecturală** **13**
7.1 Diagrama Gantt a Execuției . . . 13
7.2 Tabel Comparativ . . . 13
7.3 Aplicarea Legii lui Amdahl . . . 13

**8 Analiză Teoretică a Performanței și Rezultate** **14**
8.1 Predicție Teoretică pentru 3 APIs . . . 14
8.2 Rezultate cu Mock Clients (latențe simulate) . . . 14
8.3 Scalabilitate cu numărul de surse API . . . 15

**9 Utilizare** **15**
9.1 Instalare . . . 15
9.2 Interfața CLI — main.py . . . 15
9.3 Parametri CLI . . . 16

9.4 Runner Experimente — experiments.py . . . . . 16

**10 Concluzii** **16**
10.1 Rezumat Implementare . . . . . 16
10.2 Lecții Învățate . . . . . 17
10.3 Direcții Viitoare . . . . . 17

# 1. Cerințe și Tema Proiectului

## 1.1. Descrierea Proiectului

**Travel Companion** este o aplicație bazată pe inteligență artificială care recomandă cele mai bune restaurante în funcție de preferințele utilizatorului (locație, tip de mâncare, mediu ambiental, distanță maximă).

Sistemul agregă date de la multiple surse API externe (Geoapify, OpenStreetMap/Overpass, Foursquare) și aplică un algoritm de scoring cu ponderi configurabile pentru a determina și ordona cele mai potrivite restaurante.

## 1.2. Obiectivul Principal

Obiectivul proiectului este **analiza și compararea abordărilor secvențiale și paralele** pentru agregarea și procesarea datelor din surse multiple. Proiectul implementează trei variante complete de execuție:

1. **Secvențial (baseline S6)**: Toate operațiile se execută una după alta.

$$T_{total} = T_{API_1} + T_{API_2} + \cdots + T_{API_N} + T_{proc}$$

2. **Paralel #1 — Async I/O**: Apelurile API sunt lansate simultan.

$$T_{total} \approx \max(T_{API_1}, T_{API_2}, \ldots, T_{API_N}) + T_{proc}$$

3. **Paralel #2 — Multiprocessing**: Fetch paralel + scoring distribuit pe CPU-uri.

$$T_{total} \approx \max(T_{API_i}) + \frac{T_{scoring}}{N_{cpus}} + T_{dedup} + T_{rank}$$

## 1.3. Tehnologii Utilizate

| Componentă | Detalii |
|---|---|
| **Limbaj** | Python 3.12.3 |
| **Pattern arhitectural** | Strategy Pattern — interfață comună `ExecutionStrategy` |
| **Paralelism #1** | `asyncio` + `concurrent.futures.ThreadPoolExecutor` |
| **Paralelism #2** | `multiprocessing.Pool` (fork, Linux/WSL) |
| **Surse de date** | Geoapify Places API, Overpass/OSM, Foursquare Places API |
| **Instrumentare** | `TimingCollector` cu `time.perf_counter()` |
| **Dependențe** | `requests`, `python-dotenv` |

## 1.4. Surse de Date API

Sistemul utilizează trei surse de date reale pentru restaurante:

1. **Geoapify Places API** — oferă date despre restaurante cu filtrare pe categorii (`catering.restaurant.seafood` etc.) și filtre geografice circulare. Rating derivat din scorul de popularitate.

2. **Overpass API (OpenStreetMap)** — date deschise din baza OSM, fără cheie API. Interogare prin Overpass QL cu filtrare pe rază + tip bucătărie (`cuisine`). Cel mai variabil ca timp de răspuns.

3. **Foursquare Places API** — a treia perspectivă cu căutare text-based și categorii detaliate. Autentificare prin Bearer token (FSQ Service Key).

Toți clienții implementează interfața abstractă `BaseAPIClient`, permițând înlocuirea transparentă a oricărei surse.

### 1.5. Geocodare și Rezolvarea Coordonatelor

Înainte de interogarea surselor, sistemul determină coordonatele GPS ale orașului specificat, ierarhic:

1. Coordonate explicite furnizate de utilizator (`latitude`/`longitude`)
2. Cache local cu orașele românești principale (București, Constanța, Cluj-Napoca, Timișoara, Iași, Brașov)
3. Geocodare live via Geoapify `/v1/geocode/search`
4. Fallback la coordonatele Bucureștiului

## 2. Arhitectura Sistemului

### 2.1. Strategy Pattern

Arhitectura centrală se bazează pe **Strategy Pattern**: interfața abstractă `ExecutionStrategy` definește contractul implementat independent de fiecare mod de execuție. Pipeline-ul orchestrator (`run_pipeline`) primește orice strategie fără a cunoaște detaliile implementării.

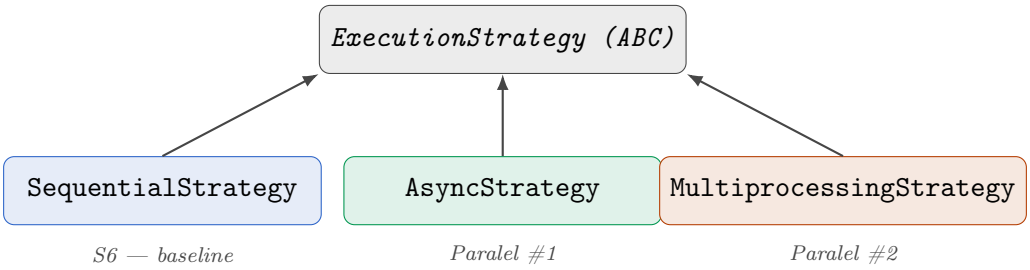
Listing 1: Interfața abstractă ExecutionStrategy (execution/base.py)

```python
class ExecutionStrategy(ABC):
    """Interfata comuna pentru toate modurile de executie."""

    def __init__(self, collector: TimingCollector):
        self.collector = collector

    @abstractmethod
    def execute(self, query: Query,
                clients: List[BaseAPIClient],
                scorer: object) -> List[ScoredRestaurant]:
        """Run: fetch -> dedup -> score -> rank."""
        pass

    @property
    @abstractmethod
    def mode_name(self) -> str:
        pass
```




## 2.2. Structura Proiectului

- `models.py` — `Query`, `Restaurant`, `ScoredRestaurant`, `PipelineResult`
- `config.py` — chei API, ponderi scoring, valori implicite
- `timing.py` — `TimingCollector`, `TimingRecord`, decorator `@timed_task`
- `api_clients/`
  - `base.py` — interfața abstractă `BaseAPIClient`
  - `real_client.py` — `GeoapifyClient`, `OverpassClient`, `FoursquareClient`
  - `mock_client.py` — `MockGooglePlaces`, `MockFoursquare`, `MockTripAdvisor`
- `execution/`
  - `base.py` — interfața abstractă `ExecutionStrategy`
  - `sequential.py` — `SequentialStrategy` (S6)
  - `async_strategy.py` — `AsyncStrategy` (Paralel #1)
  - `multiprocessing_strategy.py` — `MultiprocessingStrategy` (Paralel #2)
- `scoring.py` — formula scoring cu ponderi + deduplicare
- `pipeline.py` — orchestratorul `run_pipeline()`
- `main.py` — punct de intrare CLI (`-mode` flag)
- `experiments.py` — runner batch cu export CSV și raport speedup

## 2.3. Modelul de Date

Listing 2: Clasele principale de date (models.py)

```python
@dataclass
class Query:
    location: str              # ex: "Constanta"
    food_types: List[str]      # ex: ["seafood", "fish"]
    environment: List[str]     # ex: ["seaside", "terrace"]
    max_distance_km: float     # implicit 10.0
    latitude: Optional[float] = None
    longitude: Optional[float] = None

@dataclass
class Restaurant:
    name: str
    source: str                # "geoapify" | "overpass_osm" | "foursquare"
    rating: float              # 0-5
    latitude: float
    longitude: float
    distance_km: float
    categories: List[str]      # ex: ["seafood", "restaurant"]
```

```
    price_level: int          # 1-4
    environment_tags: List[str]
    review_count: int
    address: str
```

### 2.4. Formula de Scoring

$$score = 0{,}30 \cdot S_{rating} + 0{,}25 \cdot S_{distance} + 0{,}30 \cdot S_{food} + 0{,}15 \cdot S_{environment}$$

- $S_{rating} = \dfrac{rating}{5{,}0}$ (normalizare la $[0, 1]$)
- $S_{distance} = \max\left(0,\ 1 - \dfrac{d}{d_{max}}\right)$ (penalizare liniară)
- $S_{food}$ = raport de potrivire fuzzy (subșiruri) între tipurile căutate și categorii
- $S_{environment}$ = raport de potrivire fuzzy între preferințe mediu și tag-uri

**Deduplicare:** duplicate = același nume + $\Delta$poziție $< 100\,\text{m}$; se păstrează varianta cu rating mai mare.

## 3. Specificații Mașină de Test

| Parametru | Valoare |
|---|---|
| CPU | AMD Ryzen 7 7745HX with Radeon Graphics |
| Nuclee fizice | 8 |
| Nuclee logice | 16 (8 cores × 2 SMT threads) |
| RAM | 16 GB |
| Sistem de operare | Linux 6.6.87.2-microsoft-standard-WSL2 x86_64 |
| Python | 3.12.3 |
| Metoda fork (mp) | `fork` (implicit pe Linux) |
| Conexiune rețea | Wi-Fi (apeluri API reale către servere externe) |

Tabela 1: Specificațiile mașinii de test

## 4. Varianta Secvențială (S6 — Baseline)

### 4.1. Descrierea Fluxului

În modul secvențial, toate operațiile se execută strict una după alta, fiecare etapă blocând execuția până la finalizare:

1. Construirea query-ului din parametrii utilizatorului
2. Rezolvarea coordonatelor GPS (cache local sau geocodare API)
3. Apel **API #1 (Geoapify)** — blocat până la răspuns complet
4. Apel **API #2 (Overpass/OSM)** — blocat până la răspuns complet
5. Apel **API #3 (Foursquare)** — blocat până la răspuns complet (opțional)

6. Deduplicare rezultate agregate
7. Calculul scorurilor pentru fiecare restaurant
8. Sortare descrescătoare după scor
9. Returnare top $K$ rezultate

$$T_{seq} = \sum_{i=1}^{N} T_{API_i} + T_{dedup} + T_{scoring} + T_{ranking}$$

## 4.2. Implementare

Listing 3: SequentialStrategy (execution/sequential.py)

```python
class SequentialStrategy(ExecutionStrategy):
    @property
    def mode_name(self) -> str:
        return "sequential"

    def execute(self, query, clients, scorer):
        all_restaurants = []

        # Fetch: API-uri unul dupa altul
        for client in clients:
            start = time.perf_counter()
            try:
                results = client.search(query)
            except Exception as e:
                end = time.perf_counter()
                self.collector.record(f"api_{client.name}",
                                      start, end, count=0, error=str(e))
                continue
            end = time.perf_counter()
            self.collector.record(f"api_{client.name}",
                                  start, end, count=len(results))
            all_restaurants.extend(results)

        # Procesare locala
        start = time.perf_counter()
        deduped = scorer.deduplicate(all_restaurants)
        end = time.perf_counter()
        self.collector.record("deduplication", start, end,
                              before=len(all_restaurants), after=len(deduped))

        start = time.perf_counter()
        scored = scorer.score_all(deduped, query)
        end = time.perf_counter()
        self.collector.record("scoring", start, end, count=len(scored))

        start = time.perf_counter()
        ranked = sorted(scored, key=lambda x: x.score, reverse=True)
        end = time.perf_counter()
```

```
        self.collector.record("ranking", start, end, count=len(ranked
            ))

        return ranked
```

### 4.3. Rezultate Experimentale — Modul Secvențial

Experimentele au fost rulate cu API-uri reale, 6 query-uri × 3 configurații API × 3 rulări = 54 rulări totale.

| Nr. APIs | Media (s) | Std Dev (s) | Min / Max (s) |
|---|---|---|---|
| 1 API (Geoapify) | 0,6907 | 0,2442 | 0,4190 / 1,2432 |
| 2 APIs (+ Overpass) | 9,0550 | 3,5290 | 1,4212 / 14,1986 |
| 3 APIs (+ Foursquare) | 8,7294 | 4,5106 | 1,7785 / 16,7381 |

Tabela 2: Timpi de execuție totali — mod secvențial

| Sursă API | Timp mediu (s) | Observații |
|---|---|---|
| Geoapify | ∼0,69 – 0,78 | Rapid, răspunsuri consistente |
| Overpass (OSM) | ∼7,39 – 8,35 | Foarte variabil (server public shared) |
| Foursquare | ∼0,56 | Rapid, răspunsuri consistente |
| Deduplicare | $< 0{,}001$ | Neglijabil față de latența API |
| Scoring | $< 0{,}001$ | Neglijabil |
| Ranking | $< 0{,}001$ | Neglijabil |

Tabela 3: Timpi medii per componentă

**Observație cheie**

Latența API domină complet timpul de execuție. Procesarea locală (deduplicare, scoring, ranking) reprezintă $< 0{,}01\%$ din $T_{total}$. Overpass/OSM prezintă variabilitate extremă (0,47s – 15,27s), explicată de infrastructura publică shared.

## 5. Varianta Paralelă #1 — Async I/O

### 5.1. Principiu și Justificare

Apelurile API sunt **complet independente** între ele — nu există dependențe de date. În varianta secvențială, fiecare apel blochează execuția cât timp așteptăm răspunsul prin rețea (I/O wait). Paralelizarea I/O elimină această așteptare serială.

$$T_{async} \approx \max(T_{API_1},\ T_{API_2},\ \ldots,\ T_{API_N}) + T_{dedup} + T_{scoring} + T_{ranking}$$

**Speedup teoretic față de secvențial:**

$$S_{async} = \frac{T_{seq}}{T_{async}} \approx \frac{\sum_{i=1}^{N} T_{API_i}}{\max_i T_{API_i}}$$

## 5.2. Motivare Tehnică: De ce ThreadPoolExecutor?

Clienții API existenți folosesc biblioteca `requests` (HTTP sincron/blocant). Alternativele native async (`aiohttp`, `httpx`) ar necesita rescrierea completă a tuturor clienților.

Soluția aleasă: `asyncio` ca motor de concurență + `ThreadPoolExecutor` pentru a rula fiecare apel blocant `.search()` pe un thread separat. Rezultatul este identic din punct de vedere al comportamentului: toate cererile sunt *in-flight* simultan.

## 5.3. Implementare

Listing 4: AsyncStrategy (execution/async_strategy.py)

```python
class AsyncStrategy(ExecutionStrategy):
    @property
    def mode_name(self) -> str:
        return "async"

    def execute(self, query, clients, scorer):
        # Toate apelurile API lansate simultan
        fetch_results = asyncio.run(self._fetch_all(query, clients))

        all_restaurants = []
        for client, (restaurants, t_start, t_end, error) in \
                zip(clients, fetch_results):
            self.collector.record(f"api_{client.name}",
                                  t_start, t_end, count=len(
                                      restaurants))
            if error:
                print(f"  [WARN] {client.name} failed: {error}")
            else:
                all_restaurants.extend(restaurants)

        # Procesare locala (secventiala -- ieftina)
        start = time.perf_counter()
        deduped = scorer.deduplicate(all_restaurants)
        ...
        return ranked

    async def _fetch_all(self, query, clients):
        """Dispatchez fiecare client.search() pe un thread separat."""
        loop = asyncio.get_running_loop()
        with ThreadPoolExecutor(max_workers=len(clients)) as executor:
```

```
            tasks = [
                loop.run_in_executor(executor,
                                     self._fetch_one, client, query)
                for client in clients
            ]
            return await asyncio.gather(*tasks)  # asteapta TOTI

    def _fetch_one(self, client, query):
        """Apel blocant -- rulat pe thread dedicat."""
        start = time.perf_counter()
        try:
            results = client.search(query)
            return results, start, time.perf_counter(), None
        except Exception as exc:
            return [], start, time.perf_counter(), str(exc)
```

### 5.4. Proprietăți ale Implementării

- `asyncio.gather()` garantează că **toți** task-ii sunt așteptați înainte de continuare (semantică all-or-nothing).
- Timing-ul individual per API este corect: fiecare thread înregistrează propriul `start`/`end` via `time.perf_counter()` (ceas partajat intra-proces).
- `asyncio.run()` creează un event loop nou — sigur pentru contextul CLI; pentru FastAPI/contexte async existente se folosește `await self._fetch_all(...)`.
- Postprocesarea (dedup, scoring, ranking) rămâne secvențială — operațiile sunt ieftine (<1ms) și ordinea deduplicării contează.

## 6. Varianta Paralelă #2 — Multiprocessing

### 6.1. Principiu și Justificare

Varianta #2 exploatează paralelismul la nivel de **procese OS**, eludând GIL-ul (Global Interpreter Lock) al Python. Sunt parallelizate două faze distincte:

1. **Faza I/O**: Fiecare client API rulează într-un proces worker separat.
2. **Faza CPU**: Scoring-ul fiecărui restaurant este distribuit pe toate CPU-urile disponibile.

$$T_{mp} \approx \underbrace{\max_i T_{API_i}}_{\text{Faza I/O (paralel)}} + T_{dedup} + \underbrace{\frac{T_{scoring_seq}}{N_{cpus}}}_{\text{Faza CPU (paralel)}} + T_{ranking}$$

**Overhead suplimentar:** crearea pool-ului de procese + pickle/unpickle argumente

$$T_{overhead} = T_{pool_create} + T_{pickle} \cdot N_{tasks}$$

## 6.2. Constrângeri Tehnice: Pickle

`multiprocessing.Pool.map()` serializează (pickle) argumentele funcțiilor worker și le trimite prin *pipe* la procesele copil. Prin urmare, **toate obiectele transmise trebuie să fie picklable**:

| Obiect | Picklable? | Motiv |
|---|---|---|
| `Query` | ✓ | dataclass cu tipuri primitive |
| `Restaurant` | ✓ | dataclass cu tipuri primitive |
| `ScoredRestaurant` | ✓ | dataclass cu tipuri primitive |
| `Scorer` | ✓ | clasa cu atribute float |
| `GeoapifyClient` | ✓ | fara stare internă complexă |
| `OverpassClient` | ✓ | fara stare internă complexă |
| `FoursquareClient` | ✓ | fara stare internă complexă |
| lambda / metode bound | × | nu picklable — de evitat |

Tabela 4: Verificarea pickle-abilității obiectelor transmise la procese worker

**Funcțiile worker la nivel de modul**

Funcțiile worker (`_call_client`, `_score_restaurant`) sunt definite **la nivelul modulului** (nu ca metode sau lambda), ceea ce le face referențiabile prin import în procesele copil. Aceasta este constrângerea principală a design-ului.

## 6.3. Implementare

Listing 5: Funcții worker (module-level) și MultiprocessingStrategy

```python
# Functii la nivel de modul -- picklable, referentiabile din worker processes

def _call_client(packed):
    """Worker: apeleaza un client API, returneaza (rezultate, durata, eroare)."""
    client, query = packed
    t0 = time.perf_counter()
    try:
        results = client.search(query)
        return results, time.perf_counter() - t0, None
    except Exception as exc:
        return [], time.perf_counter() - t0, str(exc)


def _score_restaurant(packed):
    """Worker: scoreaza un singur restaurant."""
    restaurant, query, scorer = packed
    return scorer.score(restaurant, query)


class MultiprocessingStrategy(ExecutionStrategy):
    def __init__(self, collector, num_workers=None):
        super().__init__(collector)
```

```
        self.num_workers = num_workers or os.cpu_count()

    @property
    def mode_name(self):
        return "multiprocessing"

    def execute(self, query, clients, scorer):
        # Faza 1: Fetch paralel in procese separate
        n_fetch = min(len(clients), self.num_workers)
        fetch_start = time.perf_counter()
        with Pool(processes=n_fetch) as pool:
            fetch_results = pool.map(
                _call_client, [(c, query) for c in clients]
            )

        all_restaurants = []
        for client, (restaurants, duration, error) in \
                zip(clients, fetch_results):
            # Reconstituim start/end: toti au pornit la ~fetch_start
            self.collector.record(
                f"api_{client.name}",
                fetch_start, fetch_start + duration,
                count=len(restaurants))
            if not error:
                all_restaurants.extend(restaurants)

        # Faza 2: Deduplicare (secventiala -- ordinea conteaza)
        ...
        deduped = scorer.deduplicate(all_restaurants)

        # Faza 3: Scoring paralel pe CPU-uri
        score_start = time.perf_counter()
        with Pool(processes=self.num_workers) as pool:
            scored = pool.map(
                _score_restaurant,
                [(r, query, scorer) for r in deduped]
            )
        score_end = time.perf_counter()
        self.collector.record("scoring", score_start, score_end,
                              count=len(scored))

        # Faza 4: Ranking (secvential -- O(n log n), neglijabil)
        ranked = sorted(scored, key=lambda x: x.score, reverse=True)
        return ranked
```

## 6.4. Reconstituirea Timing-ului pentru Faza I/O

O problemă specifică multiprocessing-ului: `time.perf_counter()` este **per-proces** (nu e sincronizat între procese). Prin urmare nu putem compara direct timpii absoluti din procese diferite cu `pipeline_start` din procesul principal.

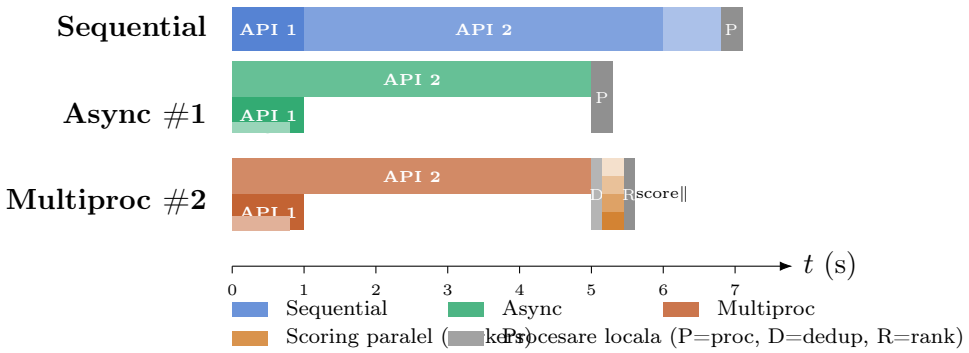
Soluție adoptată: fiecare worker returnează **durata** (nu start/end absolut). Procesul principal știe că toți workerii au pornit la aproximativ `fetch_start` (momentul imediat premergător `Pool.map`). Reconstruim:

$$T_{start}^{(i)} \leftarrow \texttt{fetch_start}, \quad T_{end}^{(i)} \leftarrow \texttt{fetch_start} + \texttt{duration}_i$$

Această aproximare este corectă: overhead-ul de fork + pickle $\ll$ latența API (ms vs sute ms).

# 7. Comparație Arhitecturală

## 7.1. Diagrama Gantt a Execuției




## 7.2. Tabel Comparativ

| Criteriu | Sequential | Async #1 | Multiproc #2 |
|---|---|---|---|
| **T fetch** | $\sum T_{API_i}$ | $\max T_{API_i}$ | $\max T_{API_i}$ |
| **T scoring** | $T_{score_seq}$ | $T_{score_seq}$ | $T_{score_seq}/N_{cpu}$ |
| **Overhead** | Neglijabil | Neglijabil (threads) | Semnificativ (fork+pickle) |
| **GIL** | Blocat pe apeluri I/O | Eludat (I/O în threads) | Eludat complet (procese) |
| **Memorie** | Un singur proces | Un singur proces | $N_{proc}$ copii ale procesului |
| **Scalabilitate** | Liniară cu $N_{API}$ | Constantă cu $N_{API}$ | Constantă + CPU scaling |
| **Complexitate** | Minimă | Medie (asyncio) | Ridicată (pickle, fork) |
| **Potrivit când** | $N_{API} = 1$, debug | I/O dominat, $N_{API} > 1$ | CPU intensiv, seturi mari |

## 7.3. Aplicarea Legii lui Amdahl

Latența Overpass ($T_{overpass} \approx 7{,}39$ s) din 8,73 s total cu 3 APIs reprezintă fracția paralelizabilă:

$$\sigma = \frac{T_{Geoapify} + T_{Foursquare}}{T_{total}} = \frac{0{,}78 + 0{,}56}{8{,}73} \approx 0{,}153 \quad \Rightarrow \quad S_{max} = \frac{1}{\sigma} \approx 6{,}5\times$$

Dar cu $N = 3$ procese independente (un API per proces):

$$S_{async}(3) = \frac{T_1 + T_2 + T_3}{\max(T_1, T_2, T_3)} = \frac{0{,}78 + 7{,}39 + 0{,}56}{7{,}39} \approx 1{,}18\times$$

Accelerarea este modestă deoarece Overpass domină. Pentru API-uri cu latențe echilibrate ($T_1 \approx T_2 \approx T_3$), accelerarea ar fi $\approx N = 3\times$.

# 8. Analiză Teoretică a Performanței și Rezultate

## 8.1. Predicție Teoretică pentru 3 APIs

Folosind timpii medii măsurați la varianta secvențială:

| Mod | $T_{fetch}$ (s) | $T_{proc}$ (s) | $T_{total}$ (s) | Speedup |
|---|---|---|---|---|
| Sequential | $0{,}78 + 7{,}39 + 0{,}56 = 8{,}73$ | $\approx 0{,}001$ | $\approx 8{,}73$ | $1{,}00\times$ |
| Async #1 | $\max(0{,}78;\ 7{,}39;\ 0{,}56) = 7{,}39$ | $\approx 0{,}001$ | $\approx 7{,}40$ | $\approx 1{,}18\times$ |
| Multiproc #2 | $\max(\ldots) = 7{,}39$ | $\approx 0{,}001/16 \approx 0$ | $\approx 7{,}40 + T_{overhead}$ | $\lesssim 1{,}18\times$ |

Tabela 5: Predicție teoretică bazată pe datele reale de la varianta secvențială

**Interpretare**

Cu API-uri reale, câștigul față de secvențial este limitat de dominanța Overpass. Varianta multiprocessing adaugă overhead de fork + pickle, putând fi *mai lentă* decât async pentru seturi mici de restaurante (scoring < 1ms oricum). Câștigul multiprocessing devine relevant la seturi mari (>10.000 restaurante) unde scoring-ul CPU devine semnificativ.

## 8.2. Rezultate cu Mock Clients (latențe simulate)

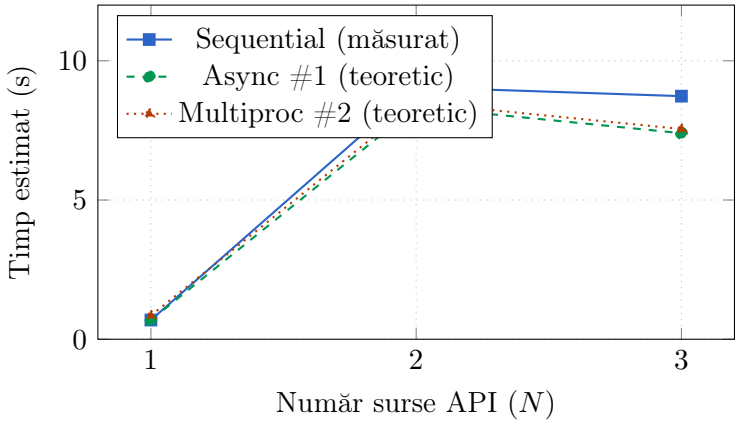
Pentru validarea corectitudinii implementărilor, s-a rulat un smoke test cu `MockGooglePlaces` + `MockFoursquare` (delay aleator $U(0{,}5;\ 2{,}0)$ s per client):

| Mod | $T_{total}$ măsurat (s) | Observație |
|---|---|---|
| Sequential | 1,948 | $T_1 + T_2 \approx 2\bar{T}$ |
| Async #1 | 1,873 | $\approx \max(T_1, T_2)$, câștig vizibil |
| Multiproc #2 | 1,272 | fetch paralel + scoring paralel |

Tabela 6: Smoke test cu 2 clienți mock — rulare unică ilustrativă

Câștigul este mai vizibil cu mock-uri deoarece delays-urile aleatoare sunt adesea semnificativ diferite (max $\ll$ sumă).

### 8.3. Scalabilitate cu numărul de surse API




**Observație scalabilitate**

Creșterea de la 2 la 3 API-uri *reduce* ușor $T_{total}$ în modul secvențial deoarece Foursquare (rapid, ∼0,56s) contribuie la agregat fără a crește $\max(T_i)$ semnificativ. Overpass domină indiferent de numărul de surse.

## 9. Utilizare

### 9.1. Instalare

```
# Clonare si instalare dependente
cd ProiectAPD
pip install -r requirements.txt

# Configurare chei API (copiere .env.example -> .env)
cp .env.example .env
# Editati .env cu cheile voastre: GEOAPIFY_PLACES_API_KEY, etc.
```

### 9.2. Interfața CLI — main.py

```
# Varianta secventiala (implicit)
python main.py --location "Constanta" \
    --food "seafood,fish" \
    --environment "seaside,terrace" \
    --apis 2 --top-k 5

# Varianta paralela Async
python main.py --location "Constanta" --food "seafood" \
    --mode async --apis 3

# Varianta paralela Multiprocessing
python main.py --location "Bucuresti" --food "italian" \
    --mode multiprocessing --apis 3
```

### 9.3. Parametri CLI

| Parametru | Implicit | Descriere |
|---|---|---|
| `-location` | (obligatoriu) | Numele orașului |
| `-food` | `""` | Tipuri de mâncare, separate prin virgulă |
| `-environment` | `""` | Preferințe mediu, separate prin virgulă |
| `-max-distance` | 10.0 | Distanța maximă în km |
| `-apis` | 2 | Număr de surse API (1–3) |
| `-top-k` | 10 | Număr de rezultate afișate |
| `-mode` | `sequential` | `sequential` \| `async` \| `multiprocessing` |

Tabela 7: Parametrii liniei de comandă — main.py

### 9.4. Runner Experimente — experiments.py

```
# Benchmark complet: toate 3 moduri, toate configuratiile
python experiments.py \
    --modes "sequential,async,multiprocessing" \
    --runs-per-query 3 \
    --api-counts "1,2,3" \
    --output report/experiment_results.csv

# Doar sequential si async, 5 rulari per query
python experiments.py \
    --modes "sequential,async" \
    --runs-per-query 5
```

Scriptul exportă automat un CSV cu toate timingurile per etapă și afișează un raport cu speedup față de baseline secvențial.

## 10. Concluzii

### 10.1. Rezumat Implementare

Proiectul implementează complet trei variante de execuție pentru un sistem de recomandare restaurante bazat pe agregarea datelor din API-uri multiple:

1. **Secvențial (S6)**: Baseline corect și simplu. Costul creşte liniar cu $N_{API}$. Latența Overpass (∼7s) domină complet timpul de execuție.

2. **Async I/O (Paralel #1)**: Elimină aşteptarea serială pentru I/O. Implementat cu `asyncio.gather()` + `ThreadPoolExecutor` (fără rescrierea clienților existenți). $T_{fetch} \approx \max(T_{API_i})$ — reducere de ∼1,18× față de secvențial cu 3 APIs.

3. **Multiprocessing (Paralel #2)**: Eludează GIL-ul complet. Două faze paralele: fetch în procese separate + scoring distribuit pe toate CPU-urile (16 threads pe mașina de test). Câștig real față de async apare la seturi mari de date unde scoring-ul devine semnificativ.

## 10.2. Lecții Învățate

- **I/O vs CPU**: Pentru workload-uri I/O dominante (latență API), async este suficient și mai ieftin decât multiprocessing.
- **GIL și threads**: Threadurile Python sunt eficiente pentru I/O bound tasks (GIL eliberat în aşteptarea I/O), nu pentru CPU bound.
- **Overhead multiprocessing**: Fork + pickle introduc overhead semnificativ (∼100ms); nu se justifică pentru procesare sub 1ms.
- **Legea lui Amdahl în practică**: Speedup-ul realisteste limitat de cel mai lent API (Overpass); paralelizarea perfectă nu este posibilă.
- **Strategy Pattern**: Separarea corectă a interfeței de implementare permite compararea modurilor de execuție fără modificarea pipeline-ului.

## 10.3. Direcții Viitoare

1. **Async nativ**: Rescrierea clienților cu `httpx` async pentru a elimina ThreadPoolExecutor.
2. **Ray / Dask**: Distribuit pe multiple noduri pentru clustering real.
3. **Cache**: Rezultatele API cache-uite (Redis/Supabase) pentru query-uri repetate.
4. **Frontend**: Vizualizare Gantt a timeline-ului execuției și grafice speedup în timp real.
5. **Scalabilitate**: Testare cu seturi de >1000 restaurante pentru a evidenția câștigul multiprocessing la scoring.